Lecture 5: Decision trees, Random forests

Max G'Sell

Machine Learning II: 46-927

February 18, 2019

Announcements

- Homework 4 is due today. Homework 5 comes out tomorrow night and is due on February 25.

Today

Last time we introduced generative models and explored LDA and some of the implications. We're going to spend a few minutes on another famous generative model, and then move on to tree-based methods.

- Remarks about the homework
- Generative model example: Naive Bayes
- Decision Trees
- Random Forests (very likely)
- Black-box diagnostics (maybe?)

Where we were

We switched from regression to classification.

Now we observe pairs $(X, Y) \sim \mathcal{P}$, with $X \in \mathbb{R}^p$ and $Y \in \{0, 1\}$ (or sometimes $\{-1, 1\}$).

Based on n observed pairs, we estimate a function $\hat{f}$ that "guesses Y well" for future $(X, Y) \sim \mathcal{P}$.

In a classification problem, we can make $K(K-1)$ different kinds of mistakes! This leads to two different $K \times K$ matrices reflecting classification performance.

Statistical decision theory

We showed that the expected misclassification error (or expected prediction error),

$$\mathrm{EPE} = \mathbb{E}\left(\mathcal{L}(Y, \hat{f}(X))\right) = \mathbb{E}\left(\mathbf{1}_{Y \neq f(X)}\right),$$

is minimized by the ideal rule:

$$f(x) \underset{g \in \{1,\ldots,K\}}{\operatorname{argmax}} \mathbb{P}(Y = g | X = x),$$

if we knew $\mathbb{P}(Y = g|X = x)$ for all g. This is called the Bayes Classifier, and the error rate it achieves is the Bayes Rate.

Picking up from last time: Building a classifier

We showed that the ideal classifier is the Bayes classifier:

$$\begin{aligned}\hat{f}(x) &= \underset{k=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(Y = k|X = x) \\ &= \underset{k=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(X = x|Y = k) \cdot \pi_k\end{aligned}$$

To build a classifier, we either:

1. Estimate $\mathbb{P}(Y = k|X = x)$ directly. (Discriminative models)
2. **Estimate both π_k and $\mathbb{P}(X = x|Y = k)$ for each class. (Generative models)**

Linear discriminant analysis

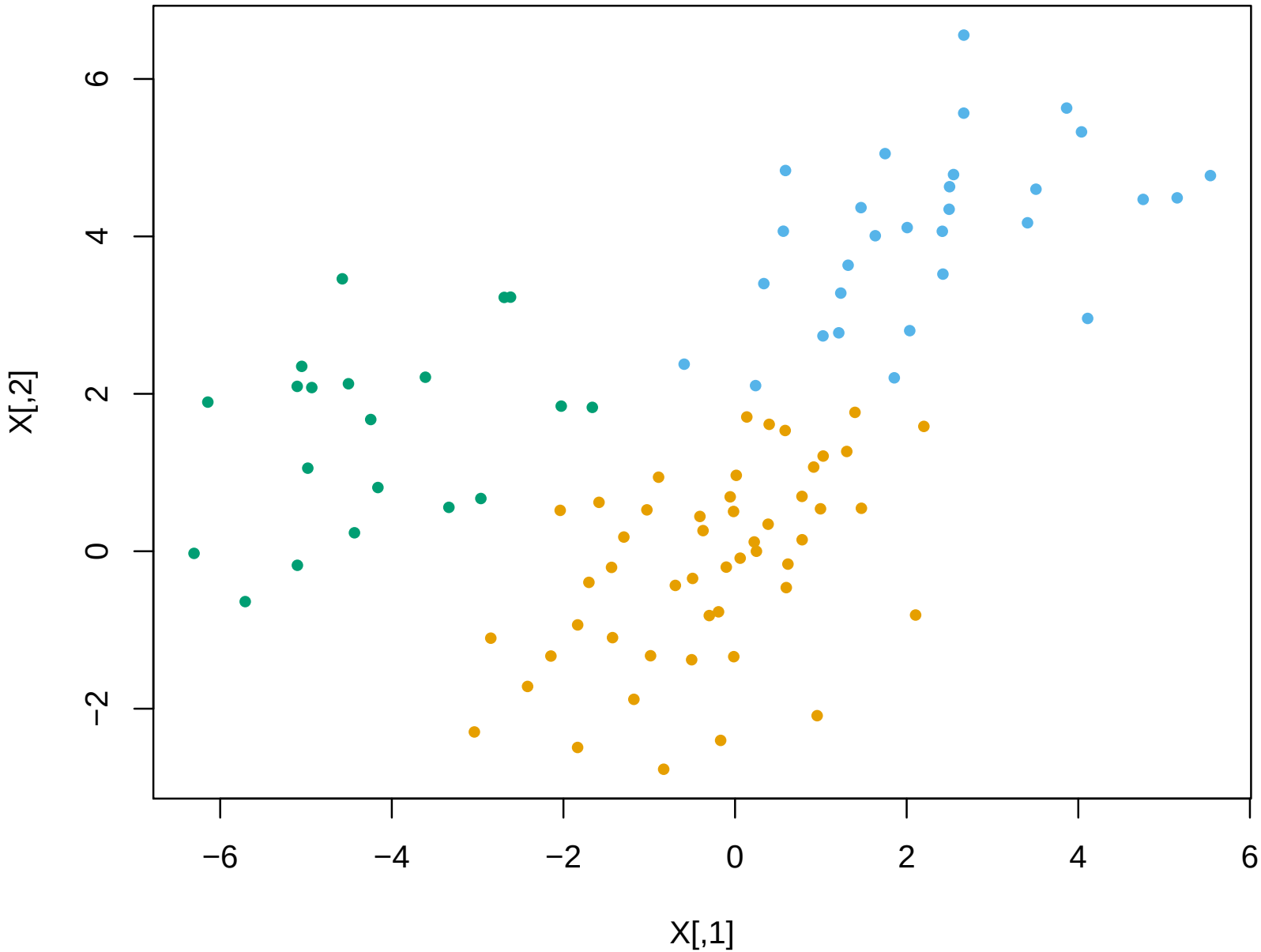
Linear discriminant analysis (LDA) models the data X within each class as being normally distributed:

$$h_j(x) = \mathrm{P}(X = x | Y = j) = N(\mu_j, \Sigma) \text{ density}$$

We allow each class to have its own mean $\mu_j \in \mathbb{R}^p$, but we assume a common covariance matrix $\Sigma \in \mathbb{R}^{p \times p}$. Hence

$$h_j(x) = \frac{1}{(2\pi)^{p/2} \det(\Sigma)^{1/2}} \exp\Big\{ -\frac{1}{2}(x - \mu_j)^T \Sigma^{-1} (x - \mu_j) \Big\}$$

Think of this as a model for classification problems where the different classes look like shifted clumps.


X[,2]
6
4
2
0
−2
−6
−4
−2
0
2
4
6
X[,1]

Modifications of LDA

Quadratic Discriminant Analysis (QDA): Each group model has a different covariance.

QDA with $\Sigma = \alpha\hat{\Sigma} + (1-\alpha)\hat{\Sigma}_k$

LDA

LDA with $\Sigma = \alpha\hat{\Sigma} + (1-\alpha)\sigma^2 I$

LDA with $\Sigma = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_p^2)$

LDA with $\Sigma = \sigma^2 I$

Nearest Shrunken Centroids

One other generative model: Naive Bayes

Imagine that you have $n = 2000$ observations and $p = 1000$ features.

It will be incredibly hard to estimate $f_k = P(X = x|Y = k)$ well for any complicated model!

You need very strong "assumptions" on f_k (reducing parameters/variance)!

Naive Bayes *assumes* (well, models) that all of the components of $X = (X_1, \ldots, X_p)$ are *independent* (conditional on Y).

Naive Bayes

Naive Bayes models all of the components of $X = (X_1, \ldots, X_p)$ as *independent* (conditional on Y).

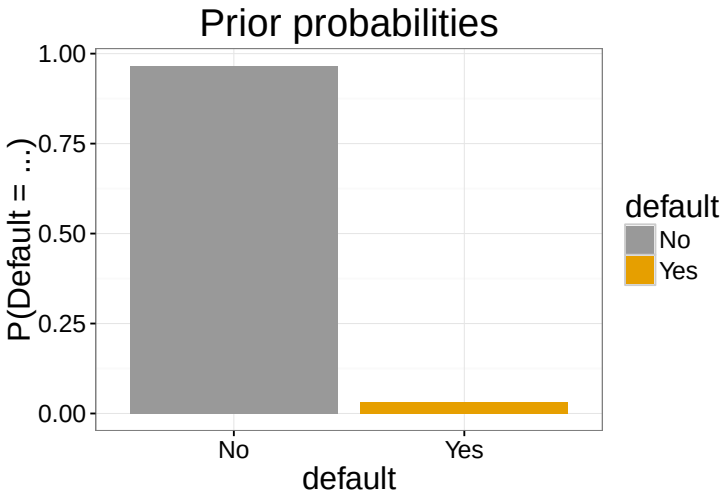
Under this independence assumption, the class distributions factor!

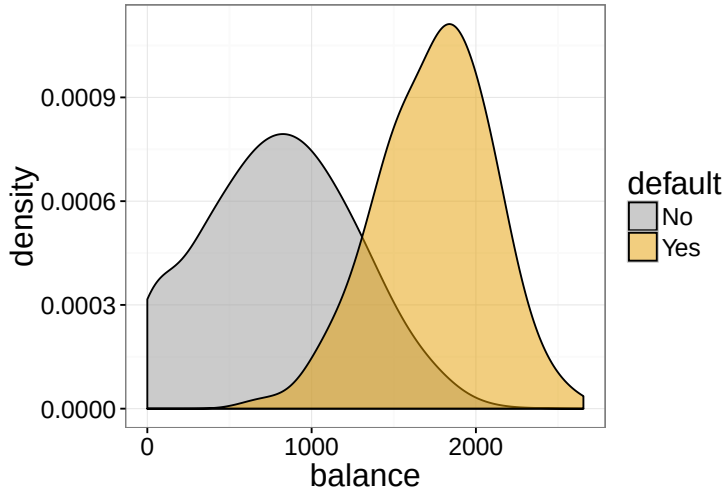
$$
\begin{aligned}
f_k(x) &= P(X = x | Y = k) \\
&= \\
&= \\
&=
\end{aligned}
$$

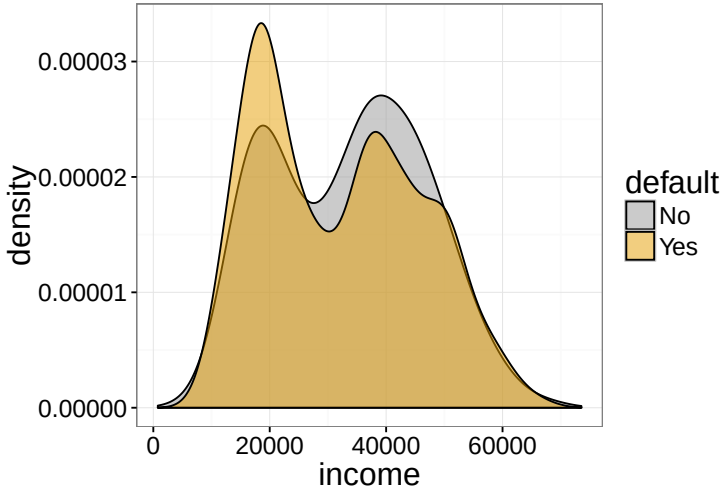
This means that we can just estimate the *univariate* distributions!

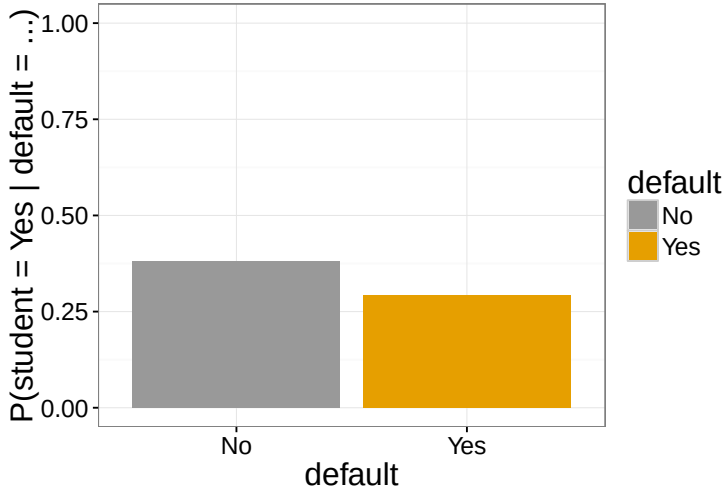
$$f_k^{(j)}(x_j) = P(X_j = x_j | Y = k)$$

Example: Default data from ISLR









We can easily calculate these simplified class distributions:

$$\hat{f}_{Yes} = \hat{f}_{Yes}(\text{income})\hat{f}_{Yes}(\text{balance})\hat{f}_{Yes}(\text{student})$$
$$\hat{f}_{No} = \hat{f}_{No}(\text{income})\hat{f}_{No}(\text{balance})\hat{f}_{No}(\text{student})$$

And then plug them into the Bayes classifier formula, just like we did for LDA

$$P(\text{default}|i, b, s) = \frac{\hat{\pi}_{Yes}\hat{f}_{Yes}(i, b, s)}{\hat{\pi}_{Yes}\hat{f}_{Yes}(i, b, s) + \hat{\pi}_{No}\hat{f}_{No}(i, b, s)}$$

Naive Bayes

Naive Bayes scales well to problems with very large p. We only need enough data to estimate each of the marginal distributions well.

It also allows you to have a flexible choice of models for each of the univariate distributions.

However, Naive Bayes cannot capture *interactions* between the features within each class!

LDA and QDA are able to incorporate these feature interactions, at the cost of needing to estimate them.

Back to discriminative models

We have two formulations of an ideal classifier:

$$\begin{aligned}\hat{f}(x) &= \underset{j=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(Y = j | X = x) \\ &= \underset{j=1,\ldots K}{\operatorname{argmax}} \ \mathrm{P}(X = x | Y = j) \cdot \pi_j\end{aligned}$$

1. **Discriminative models:** We can try to estimate $\mathbb{P}(Y = j | X = x)$ directly. (Discriminative models)

2. Generative models: We could try to estimate the distributions $\mathbb{P}(X = x | Y = j)$ for each class.

Logistic regression

We introduced logistic regression as a first example of a discriminative classifier.

We model $Y_i \sim \text{Bernoulli}(P(Y = 1|X = x))$ with

$$\text{P}(Y = 1|X = x) = \frac{e^{\beta^T x}}{1 + e^{\beta^T x}} = \frac{1}{1 + e^{-\beta^T x}}$$

and then maximize the likelihood.

However, these modeling choices can be quite restrictive. Even if we expand beyond simple linear functions, we have to specify a good functional form and the important interactions.

We will now construct an alternative set of discriminative classifiers that will circumvent these problems.

Flexible models

We've seen a couple flexible models:

- Additive models: flexible about transformations, but require interactions to be specified.
- k-nearest neighbors: can be flexible and predict well, with enough data and a good distance. However, nearly impossible to interpret! No simplification, requires all training data to make a prediction. Struggles in high dimensions.
- SVMs: (In the future, ML 2) Can capture interactions or transformations indirectly by changing kernels, but the connections are not direct and only some can be captured.

Today we are going to introduce new styles of flexible methods that avoid many of these problems.

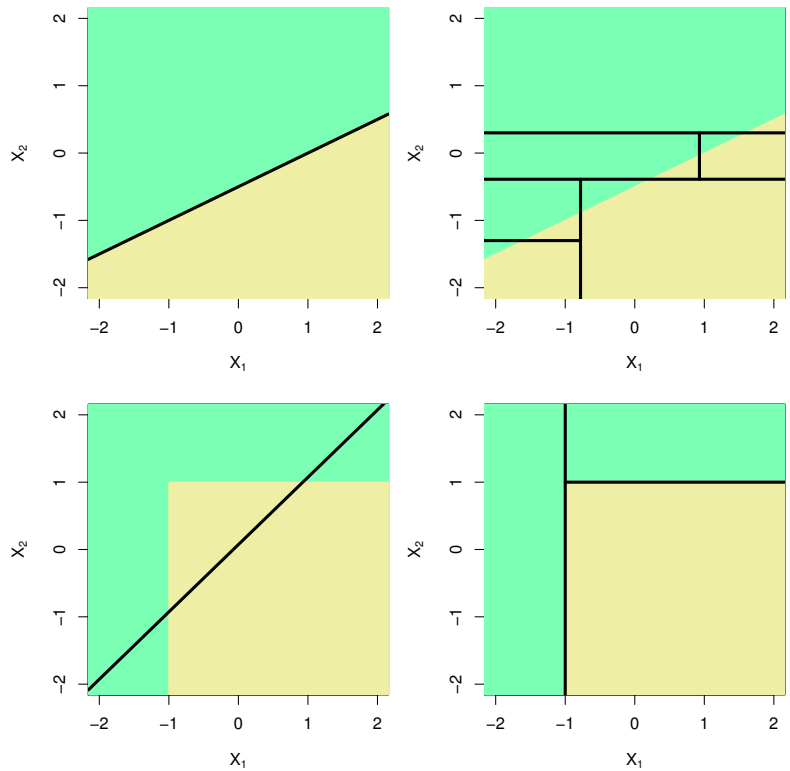
Wish list

There are many properties that we might want for a good prediction method.

- Good predictions and good probability estimates
- The ability to tune bias/variance
- Interpretability: What are our predictions based on?
- Automatic handling of variable transformations
- Automatic detection of interactions

Overview: Tree-based methods

Tree-based based methods for predicting y from a feature vector $x \in \mathbb{R}^p$ divide up the feature space into rectangles, and then fit a very simple model in each rectangle. This works both when y is discrete and continuous, i.e., both for classification and regression

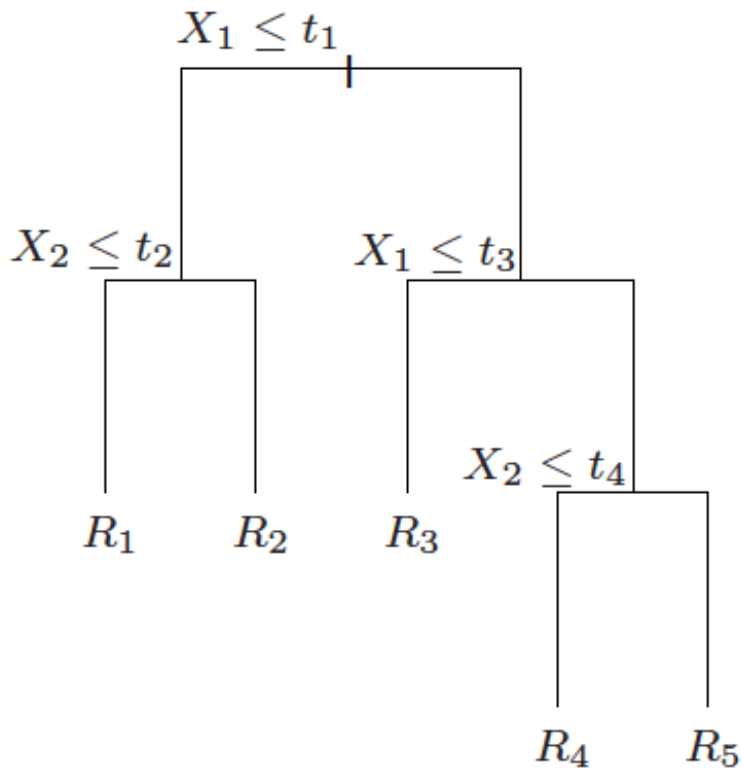


(ISL Figure 8.7)

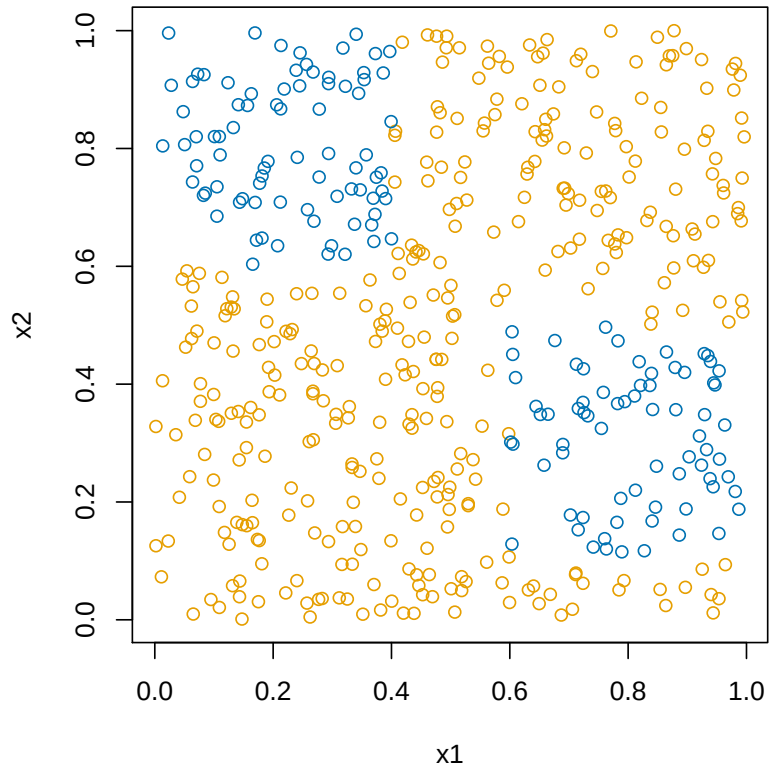
This is a big shift from thinking about linear-style models!

Tree-based methods

Tree-based methods will fix some of the problems we have encountered with other methods. They will give us a flexible rule, but one we can understand.



Overview: Tree-based methods

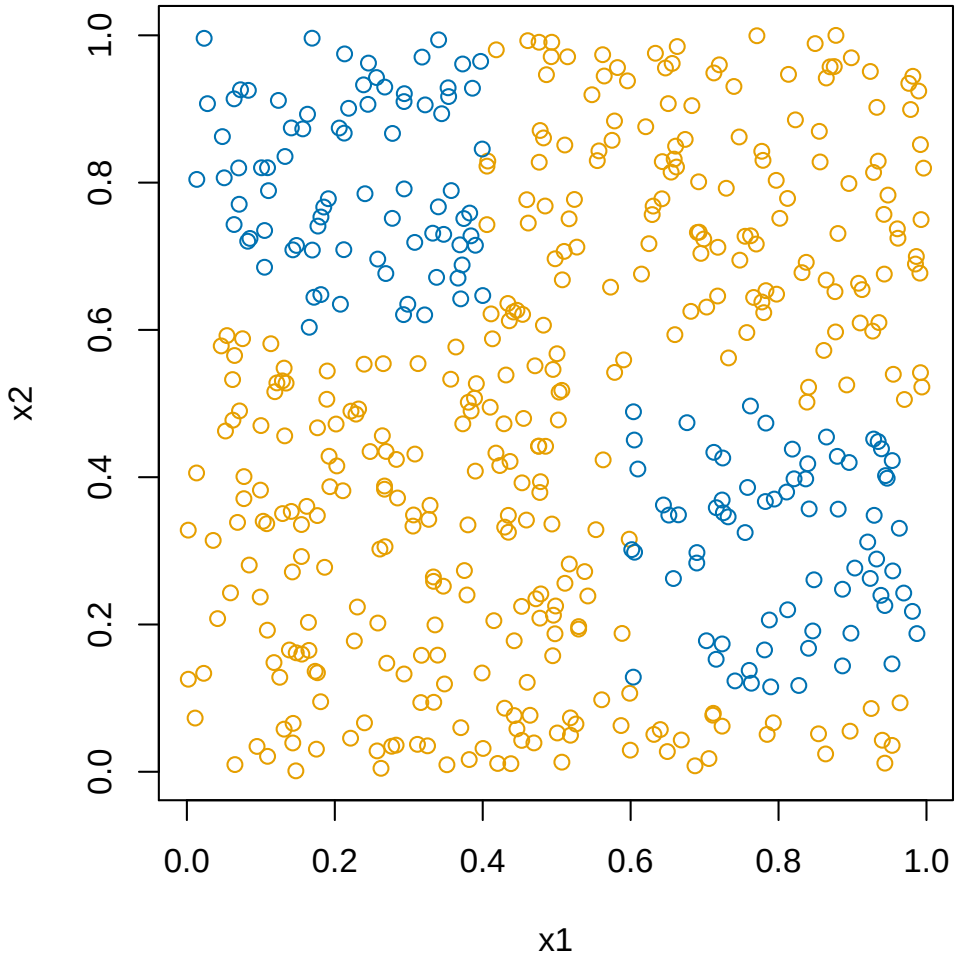


Does dividing up the feature space into rectangles look like it would work here?

Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

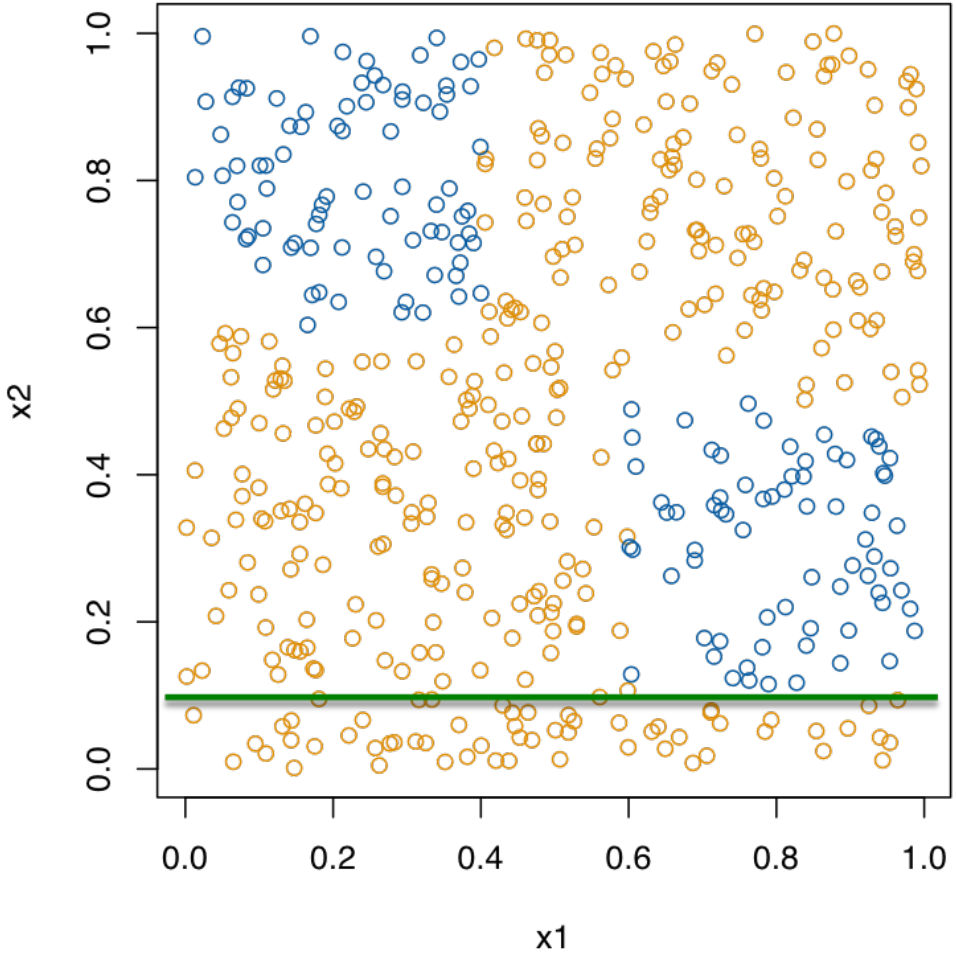
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

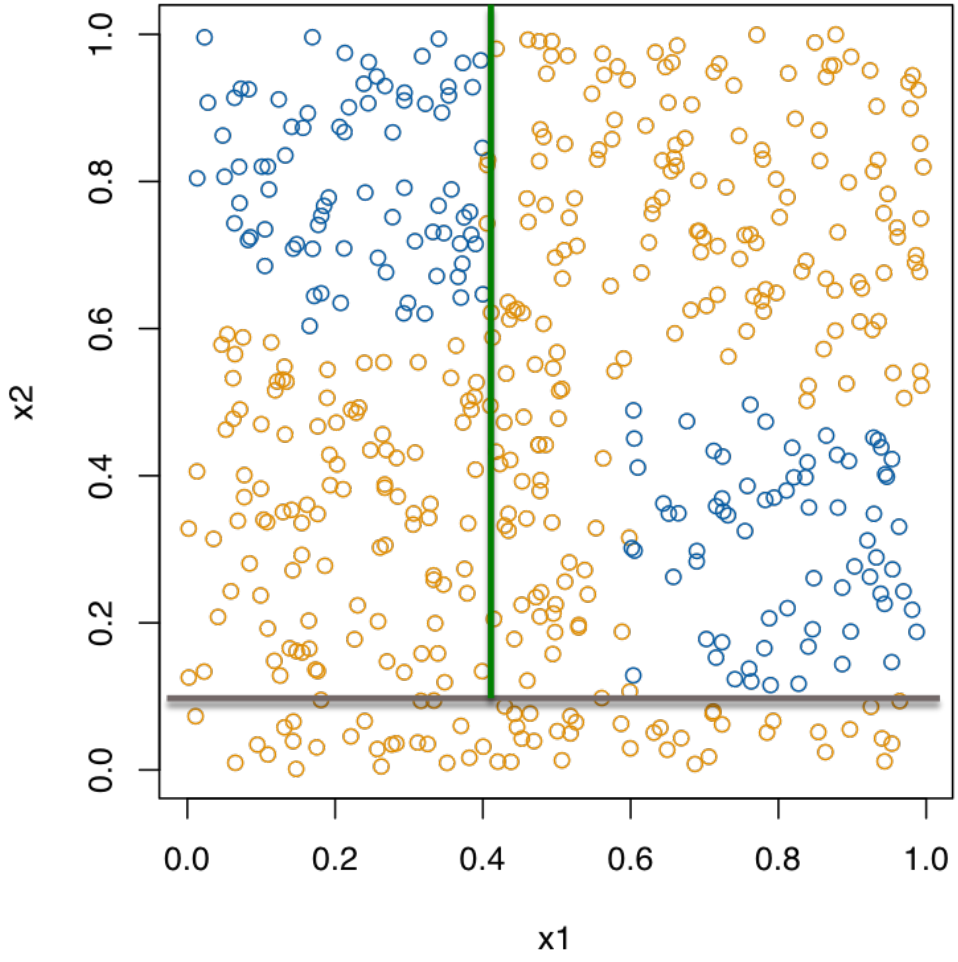
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

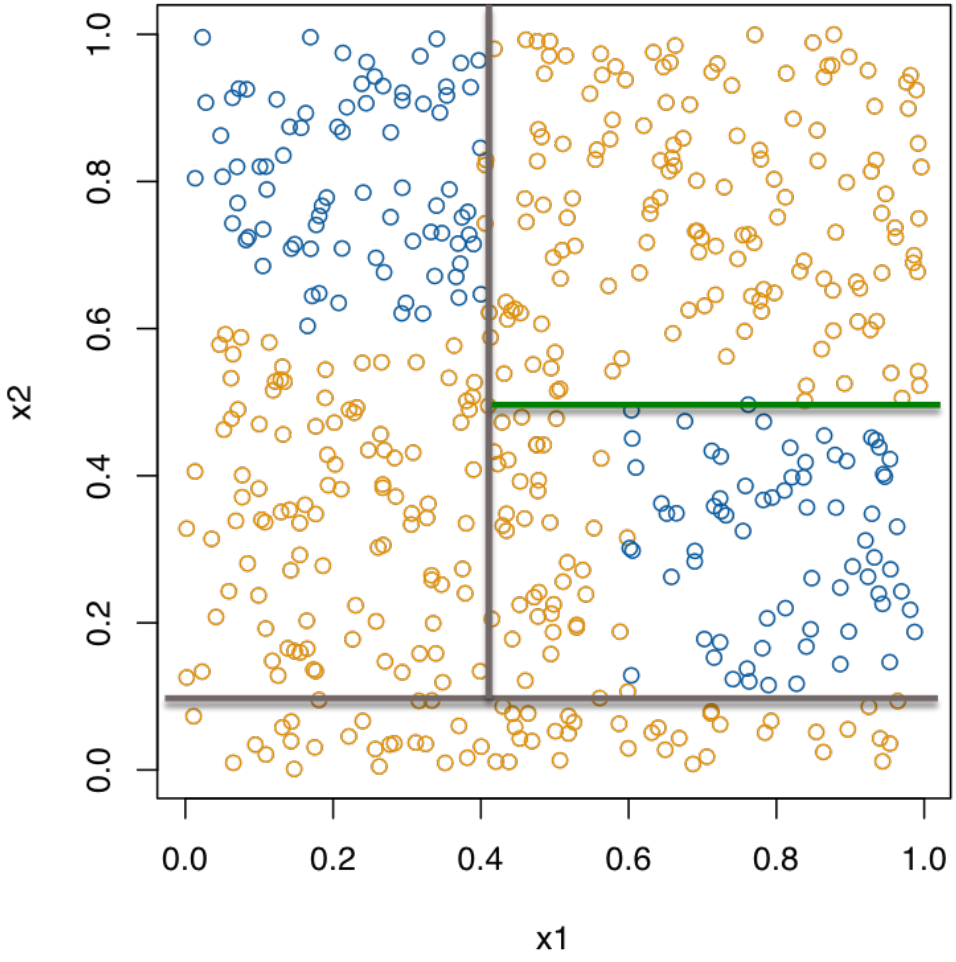
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

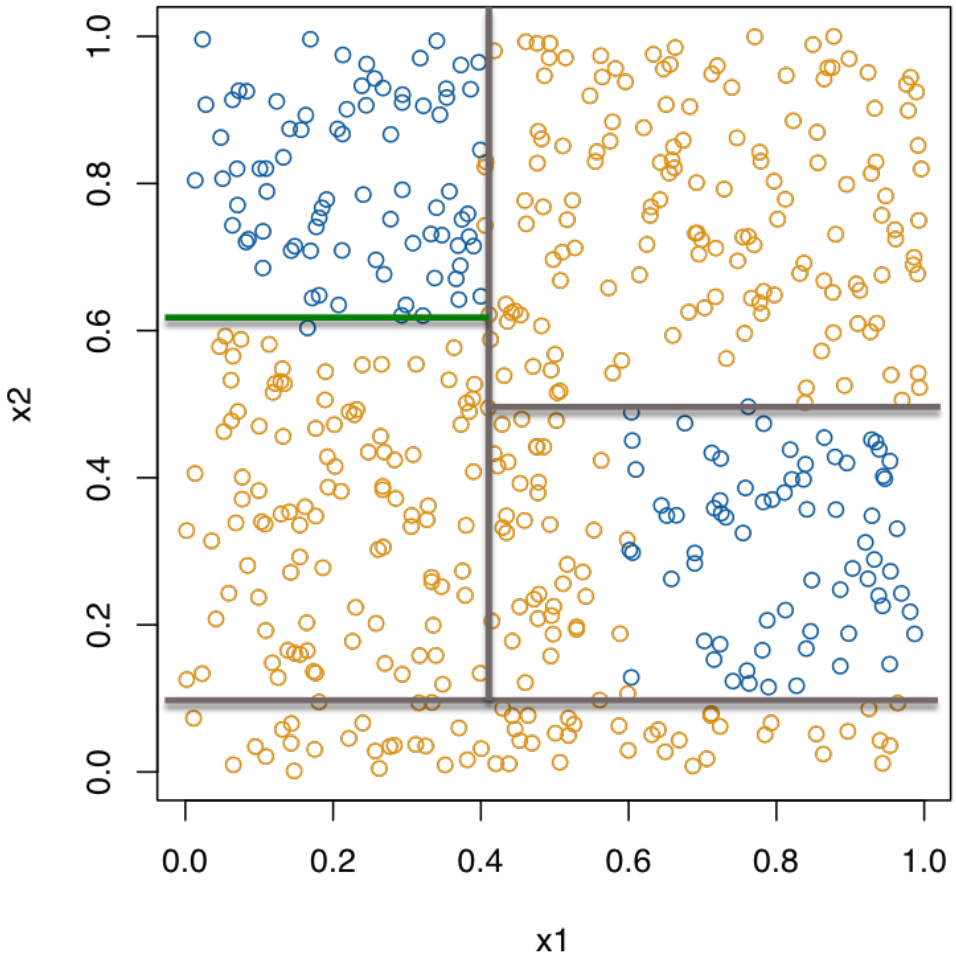
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

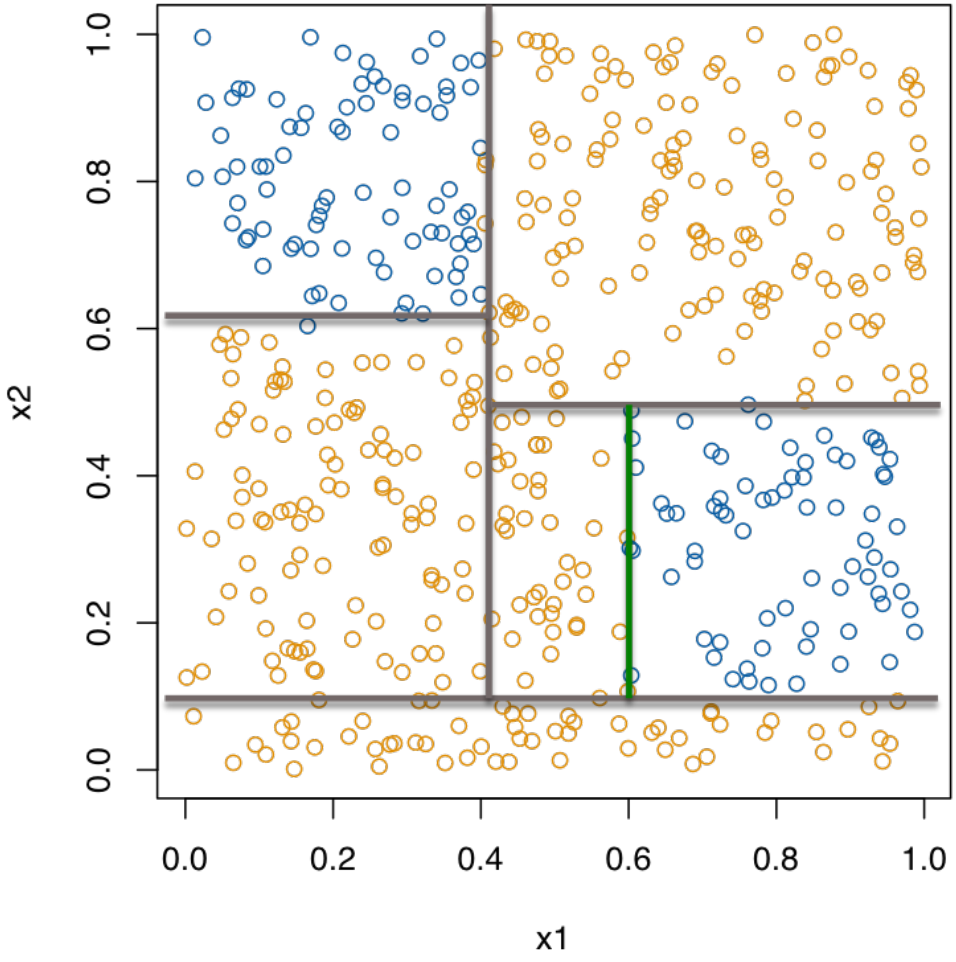
We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

Searching over all possible rectangular partitions is hard! There are many such partitions!

We make it simpler by considering a sequence of binary splits on individual variables.



Overview: Tree-based methods

We simplify our partition search by only considering a sequence of binary splits on single variables.

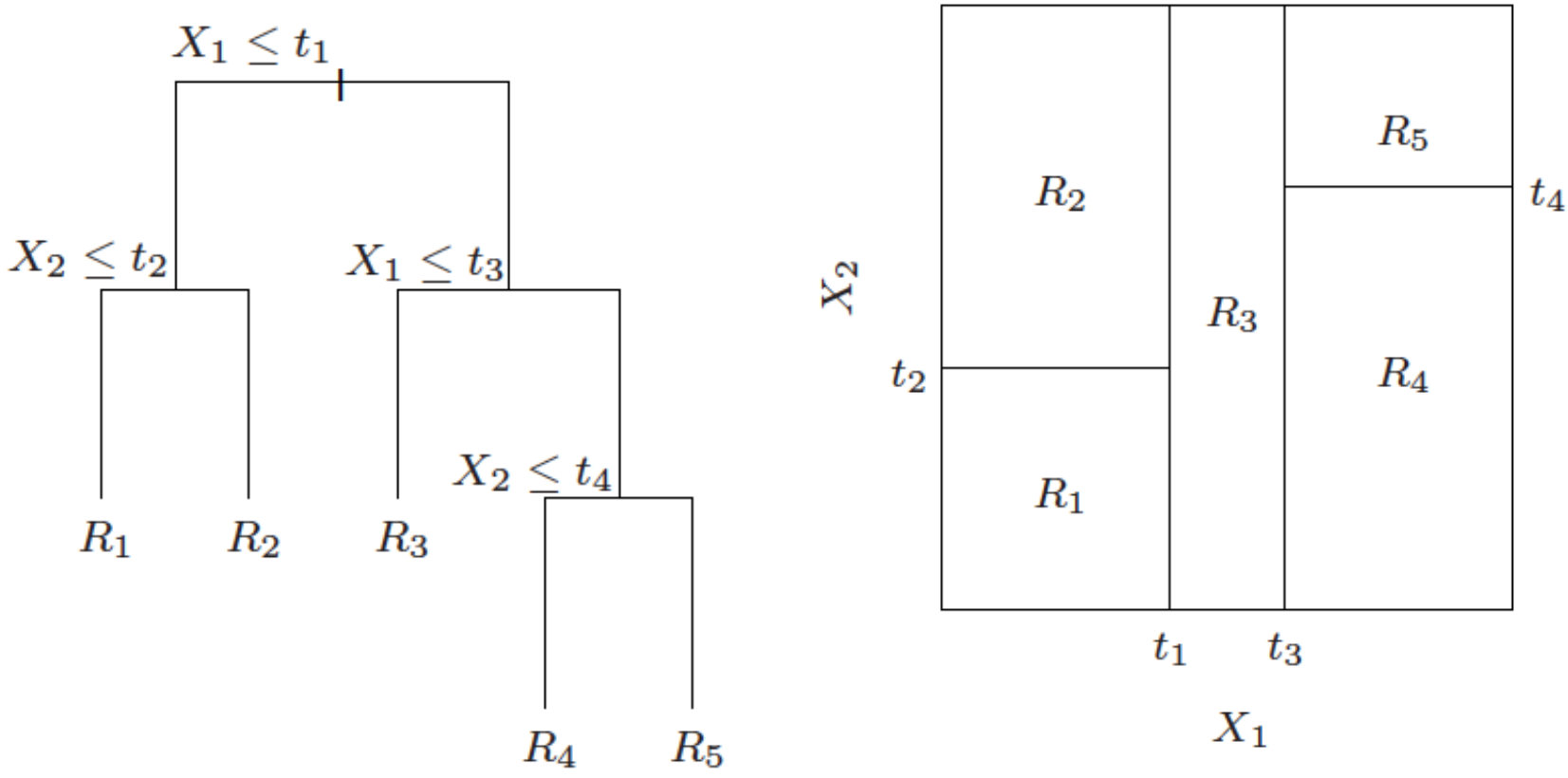
I.e., we choose a variable X_j, $j = 1, \ldots p$, divide up the feature space according to

$$X_j \leq c \quad \text{and} \quad X_j > c.$$

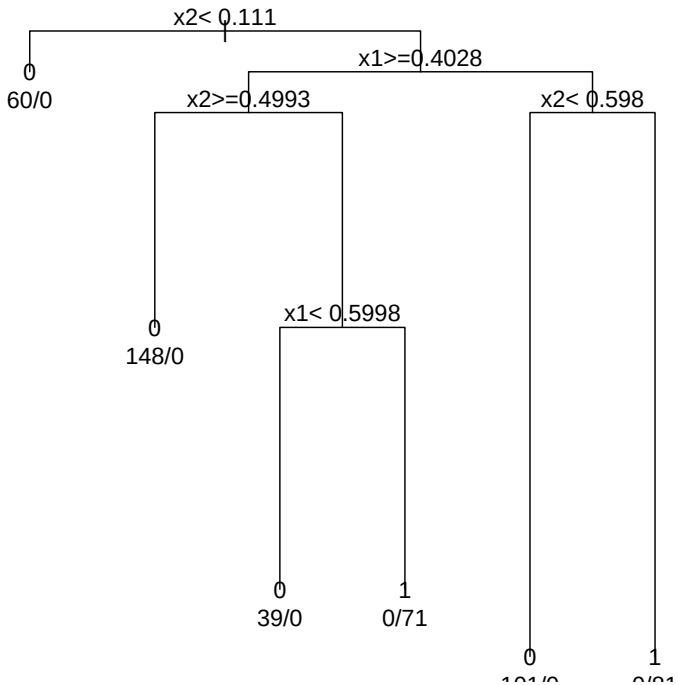
Then we proceed on each half separately.

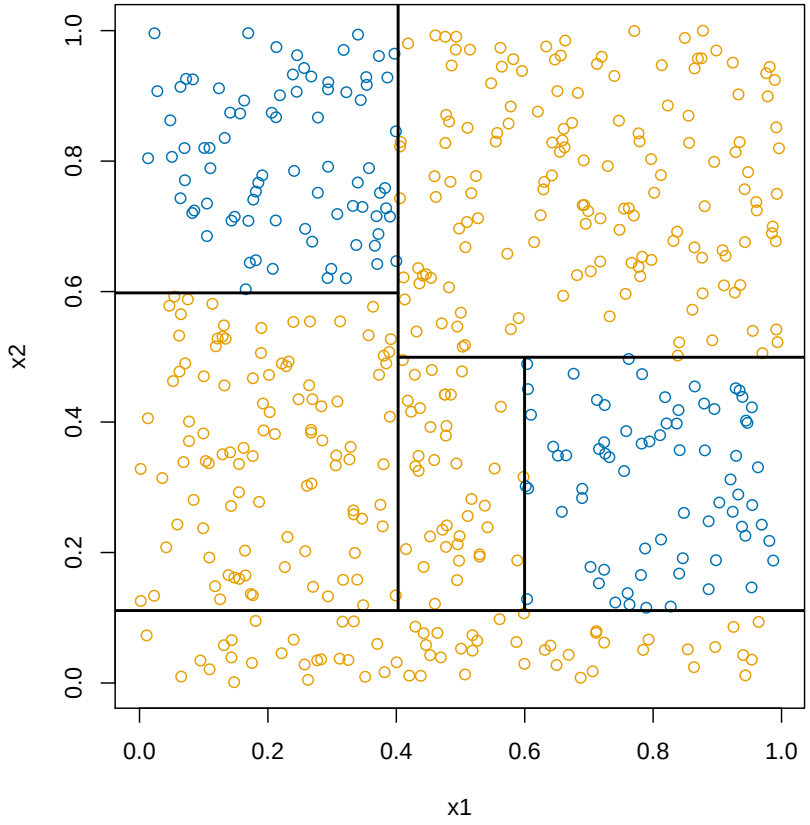
To make this even simpler, we choose the splits in a "greedy" fashion, only looking at the payoff for the current split and ignoring advantages for future splits.

A benefit of this search approach is that it also gives a natural tree representation of the partition.



(From ESL page 306)





This gives a rule that is easy to understand, easy to explain, and easy to implement!

No more coefficients to interpret!

Classification trees

Let's start with classification.

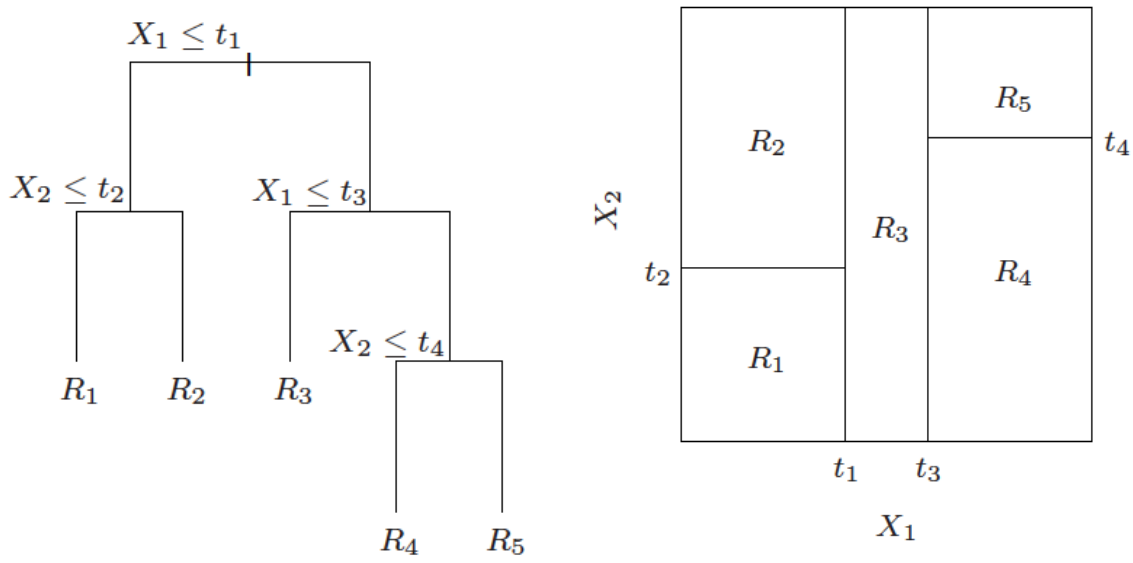
Classification trees are popular because they are interpretable, and sometimes because they mimic the way decisions are made.

Let (x_i, y_i), $i = 1, \ldots n$ be the training data, where $y_i \in \{1, \ldots K\}$ are the class labels, and $x_i \in \mathbb{R}^p$ measure the p predictor variables.

We are going to use a tree to define our classification function $\hat{f}(x) : \mathbb{R}^p \to \{1, \ldots, K\}$.

Think about the tree taking the place of our linear model in previous methods.

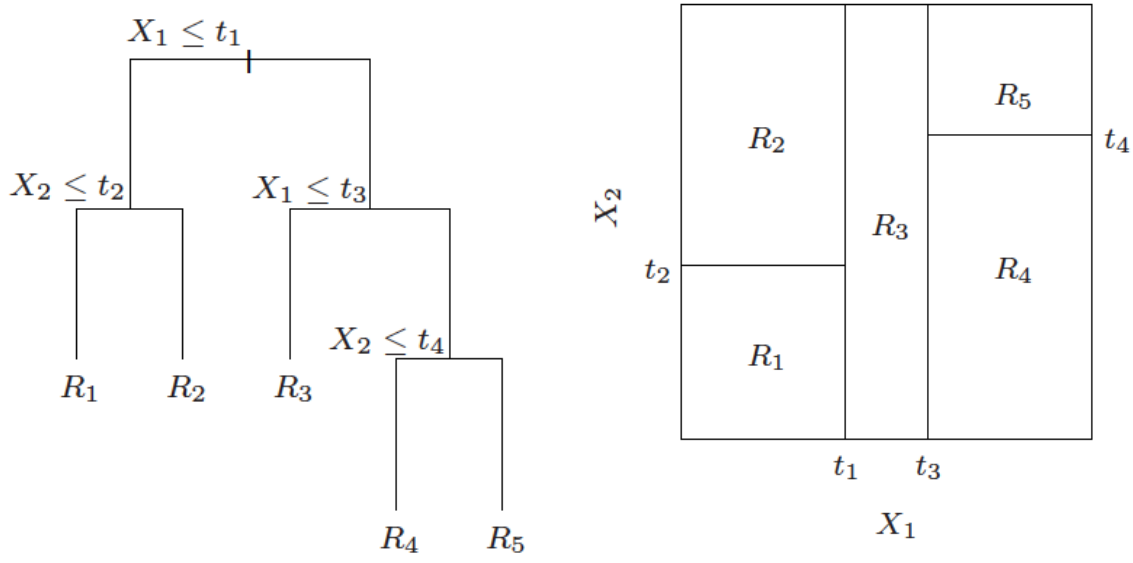
Classification trees



The classification tree can be thought of as defining m regions (rectangles) $R_1, \ldots R_m$, each corresponding to a leaf of the tree

We assign each R_j a class label $c_j \in \{1, \ldots K\}$. We then classify a new point $x \in \mathbb{R}^p$ by

$$\hat{f}^{\text{tree}}(x) \;=\; \sum_{j=1}^{m} c_j \cdot 1\{x \in R_j\} \;=\; c_j \;\text{ if }\; x \in R_j$$



$$\hat{f}^{\text{tree}}(x) = \sum_{j=1}^{m} c_j \cdot 1\{x \in R_j\}$$

Finding out which region a given point x belongs to is easy since the regions R_j are defined by a tree—we just scan down the tree. Otherwise, it would be a lot harder (need to look at each region)

Estimated class probabilities

We now have a flexible function that takes in covariates $x \in \mathbb{R}^p$ and produces classifications!

In the last few lectures, we realized that we want something more. We want actual estimates of the class probabilities!

How can we get these?

Estimated class probabilities

Note that each region R_j contains some subset of the training data (x_i, y_i), $i = 1, \ldots n$, say, n_j points.

We have been predicting class c_j using the most common class among points in R_j.

For each class k, we can also estimate the probability that a point has that class, given that it falls in R_j, $\mathrm{P}(C = k | X \in R_j)$, by

$$\hat{p}_k(R_j) = \frac{1}{n_j} \sum_{x_i \in R_j} 1\{y_i = k\}$$

the proportion of points in the region that are of class k.

We can even think of our predicted class as

$$c_j = \underset{k=1,\ldots K}{\operatorname{argmax}} \ \hat{p}_k(R_j)$$

How to build trees?

There are two main issues to consider in building a tree:

1. How to choose the splits?
2. How big to grow the tree?

Think first about varying the depth of the tree ... which is more complex/flexible, a big tree or a small tree? What tradeoff is at play here? How might we eventually consider choosing the depth?

Now for a fixed depth, consider choosing the splits. If the tree has depth d, it could have $\approx 2^d$ nodes. At each node we could choose any of p the variables for the split—this means that the number of possibilities is

$$p \cdot 2^d$$

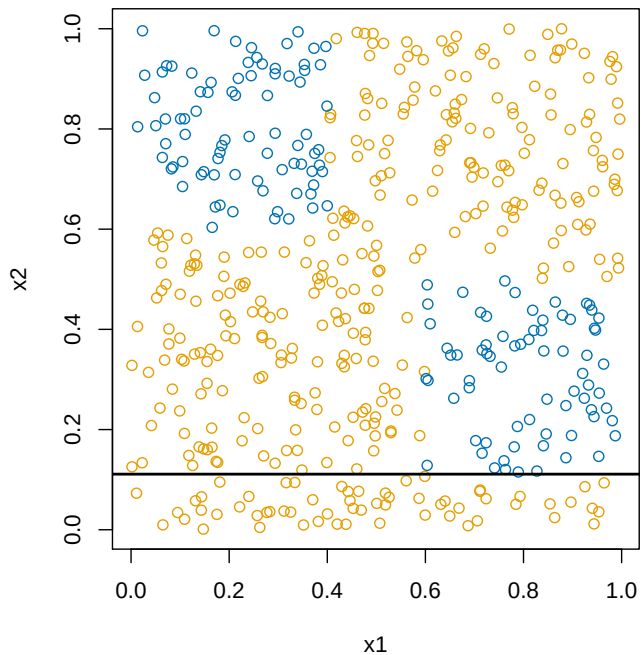
This is huge even for moderate d! And we haven't even counted the actual split points themselves

The CART algorithm

The CART algorithm[1] chooses the splits in a top down fashion: First chooses the first variable to be the root, then the variables at the second level, etc.

At each stage we choose the split to achieve the biggest drop in misclassification error—this is called a greedy strategy. In terms of tree depth, the strategy is to grow a large tree and then prune at the end.

Why grow a large tree and prune, instead of just stopping at some point?



[1] Breiman et al. (1984), "Classification and Regression Trees"

Recall that in a region R_m, the proportion of points in class k is

$$\hat{p}_k(R_m) = \frac{1}{n_m} \sum_{x_i \in R_m} 1\{y_i = k\}.$$

The CART algorithm begins by considering splitting on variable j and split point s, and defines the regions

$$R_1 = \{X \in \mathbb{R}^p : X_j \leq s\}, \quad R_2 = \{X \in \mathbb{R}^p : X_j > s\}$$

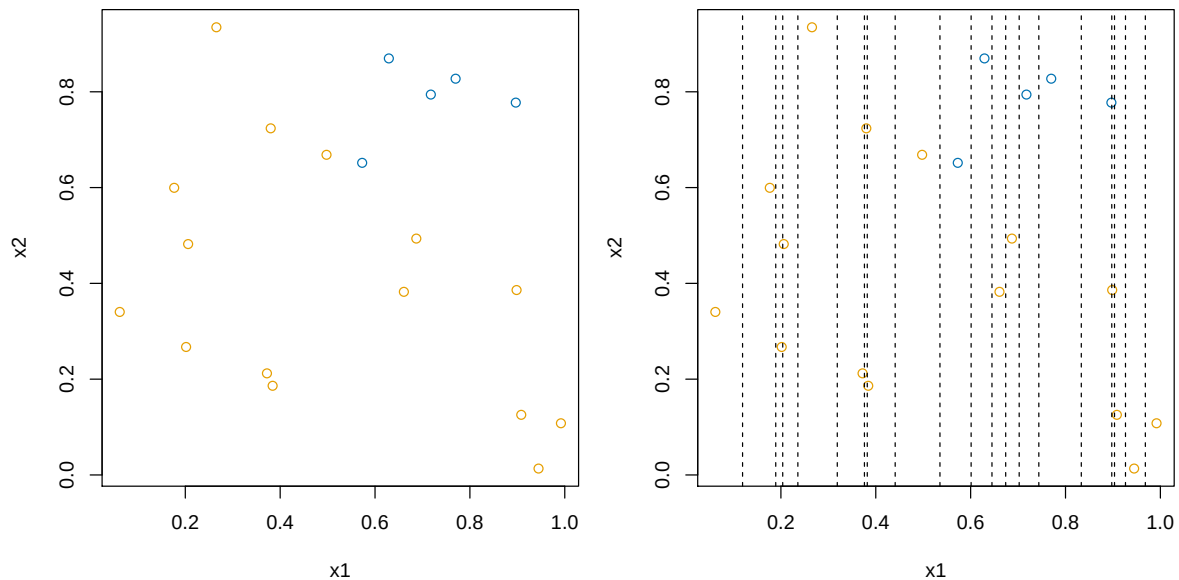
We then greedily chooses j, s by minimizing the misclassification error

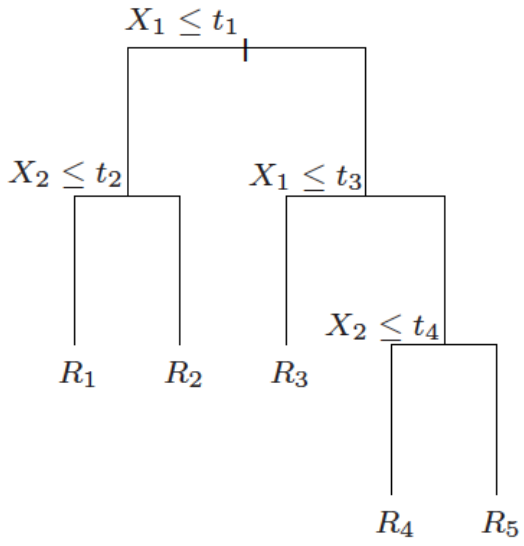
$$\underset{j,s}{\text{argmin}} \left(n_{R_1} \left[1 - \hat{p}_{c_1}(R_1)\right] + n_{R_2} \left[1 - \hat{p}_{c_2}(R_2)\right] \right)$$

Here $c_1 = \text{argmax}_{k=1,\ldots K} \, \hat{p}_k(R_1)$ is the most common class in R_1, and $c_2 = \text{argmax}_{k=1,\ldots K} \, \hat{p}_k(R_2)$ is the most common class in R_2

We now repeat this within each of the newly defined regions R_1, R_2. Again consider all variables and split points for each of R_1, R_2, greedily choosing the biggest improvement in misclassification error.

How
do we find the best split s? Aren't there infinitely many to consider?

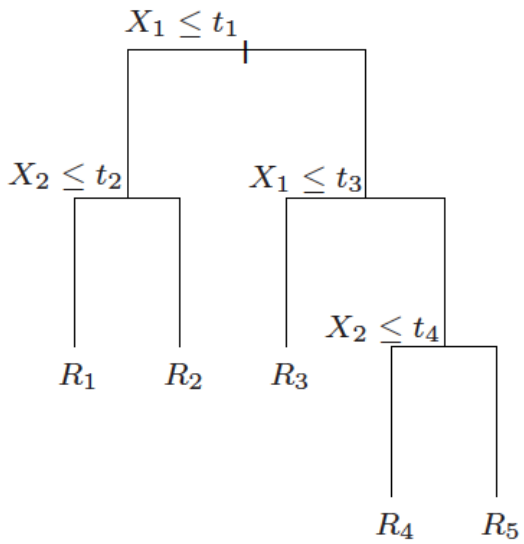




Continuing on in this manner, we will get a big tree T_0. Its leaves define regions $R_1, \ldots R_m$. We then prune this tree, meaning that we collapse some of its leaves into the parent nodes

How should we decide how much to prune a tree?

What is weird about applying this here?



For any tree T, let $|T|$ denote its number of leaves. We define

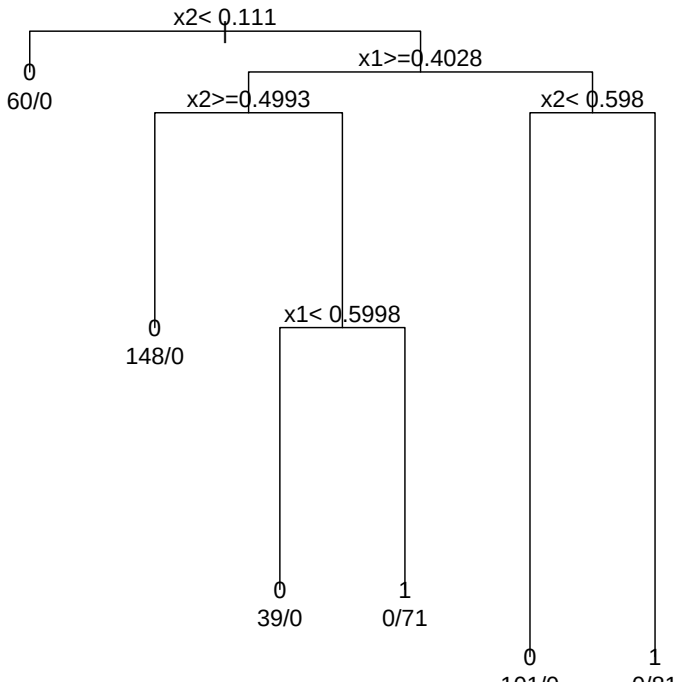
$$C_\alpha(T) = \sum_{j=1}^{|T|} \left[1 - \hat{p}_{c_j}(R_j)\right] + \alpha|T|$$

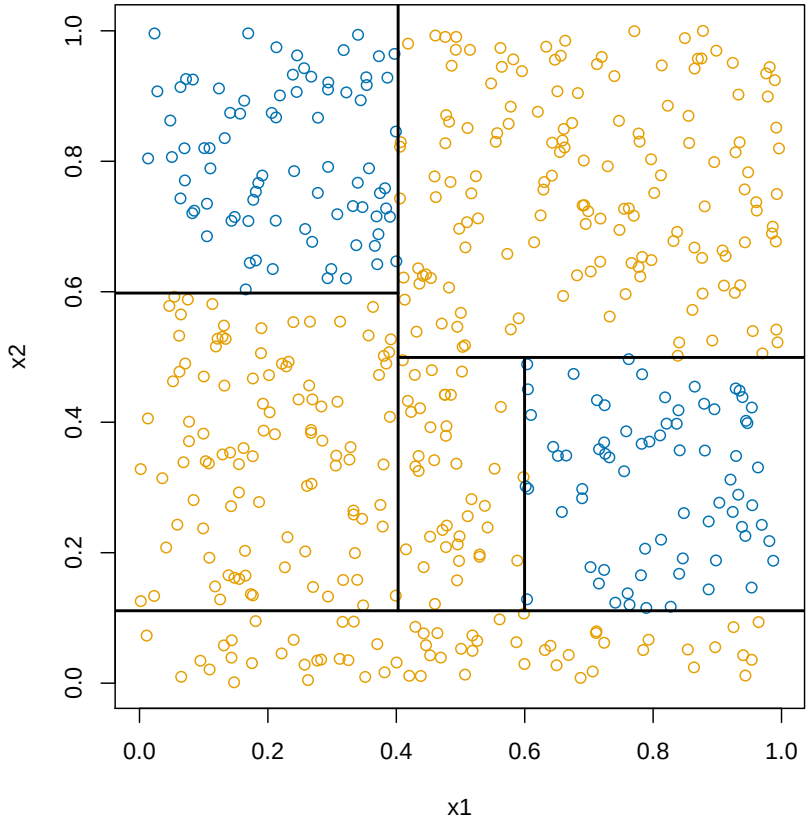
We seek the tree $T \subseteq T_0$ that minimizes $C_\alpha(T)$. It turns out that this can be done by pruning the weakest leaf one at a time.

Note that α is a tuning parameter, and a larger α yields a smaller tree. CART picks α by 5- or 10-fold cross-validation

Example: simple classification tree

Example: $n = 500$, $p = 2$, and $K = 2$. We ran CART:





In R, can use `rpart` or `tree`. In python, `DecisionTreeClassifer`. However, python does not yet support pruning (as far as I can tell)!!

Example: spam data

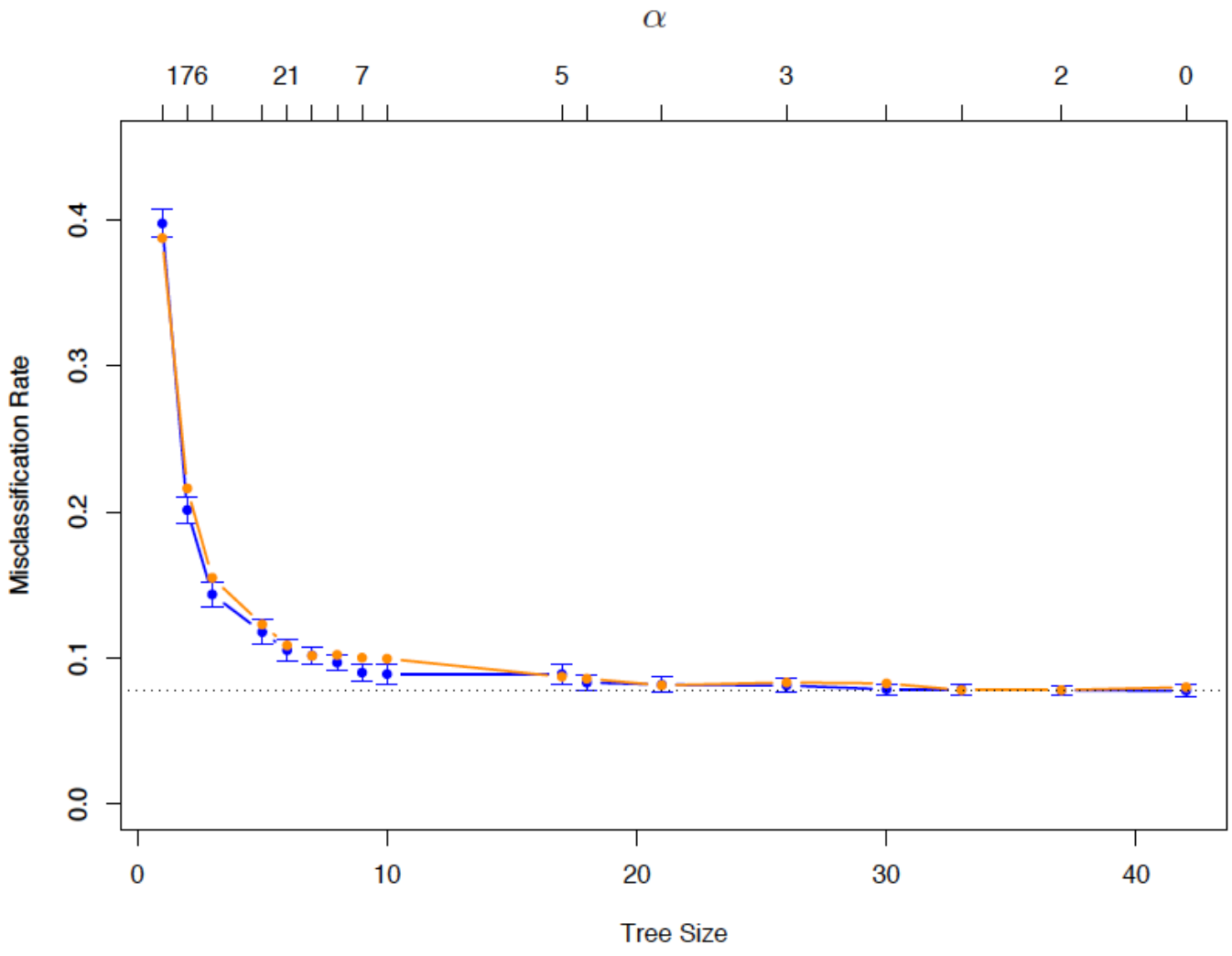
Example: $n = 4601$ emails, of which 1813 are considered spam. For each email we have $p = 58$ attributes.

The first 54 features measure the frequencies of 54 key words or characters (e.g., "free", "need", "$"). The last 3 measure

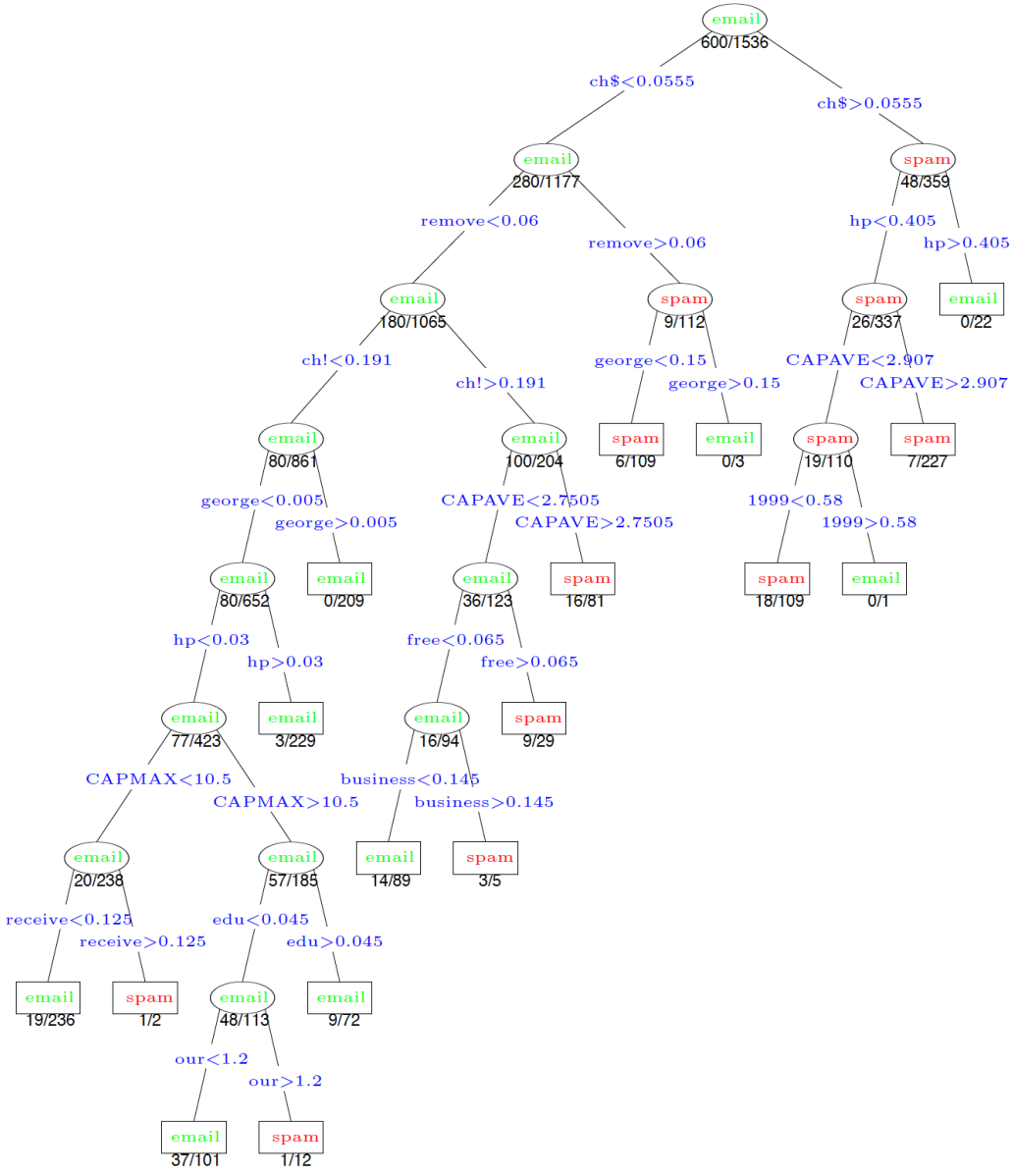
- the average length of uninterrupted sequences of capitals;
- the length of the longest uninterrupted sequence of capitals;
- the sum of lengths of uninterrupted sequences of capitals

(Data from ESL section 9.2.5)

Cross-validation error curve for the spam data (from ESL page 314):



Tree of size 17, chosen by cross-validation (from ESL page 315):



Other impurity measures

We used misclassification error as a measure of the impurity of region R_j,

$$1 - \hat{p}_{c_j}(R_j)$$

But there are other useful measures too: the Gini index:

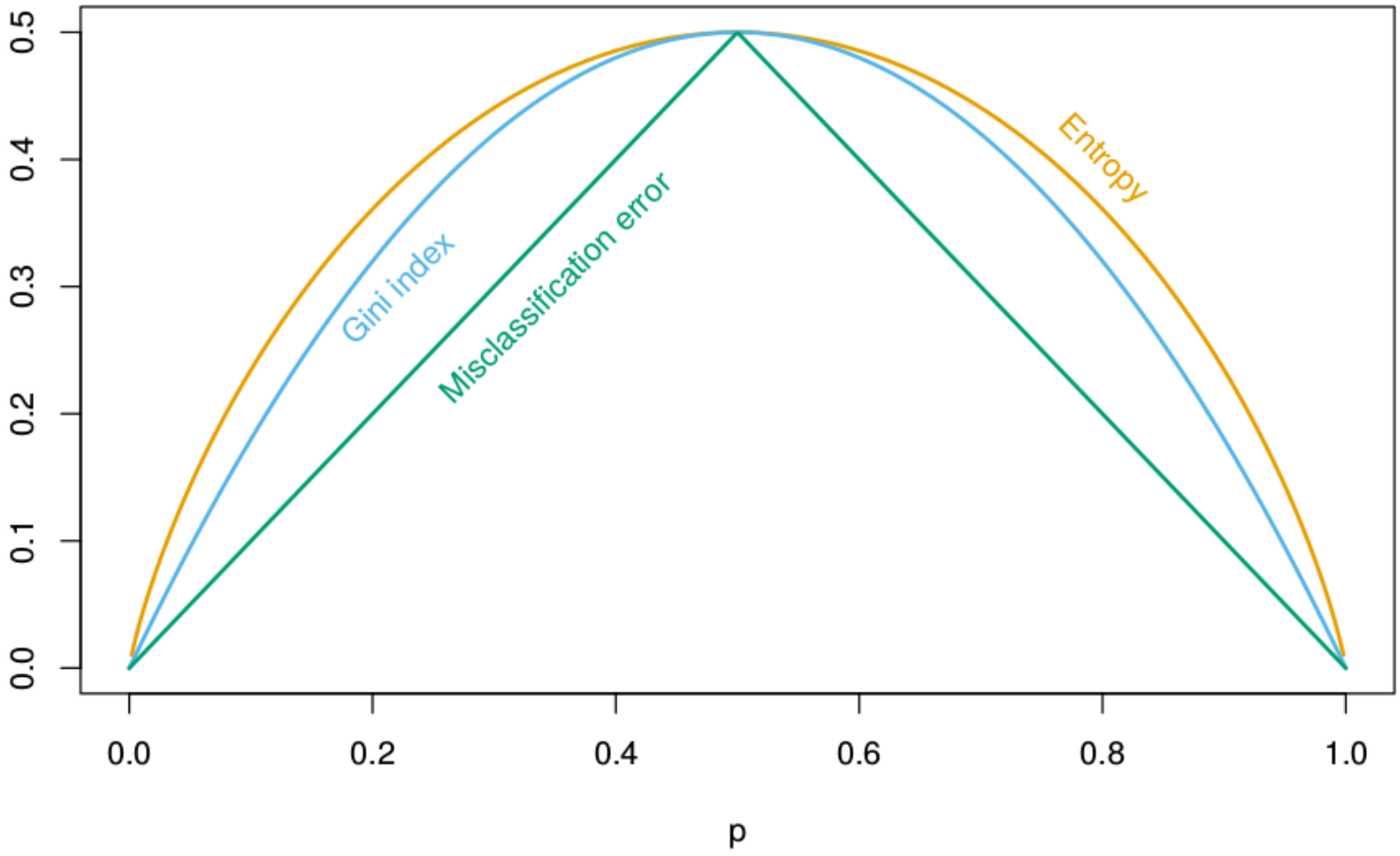
$$\sum_{k=1}^{K} \hat{p}_k(R_j)\left[1 - \hat{p}_k(R_j)\right],$$

and the cross-entropy or deviance:

$$-\sum_{k=1}^{K} \hat{p}_k(R_j) \log\left\{\hat{p}_k(R_j)\right\}.$$

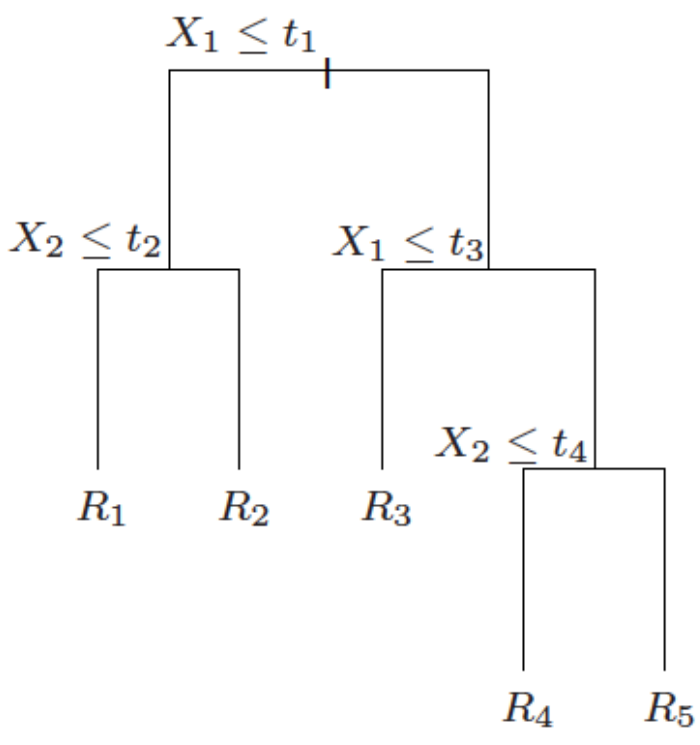
Using these measures instead of misclassification error is often better because it leads to purer nodes early on, improving the greedy search.

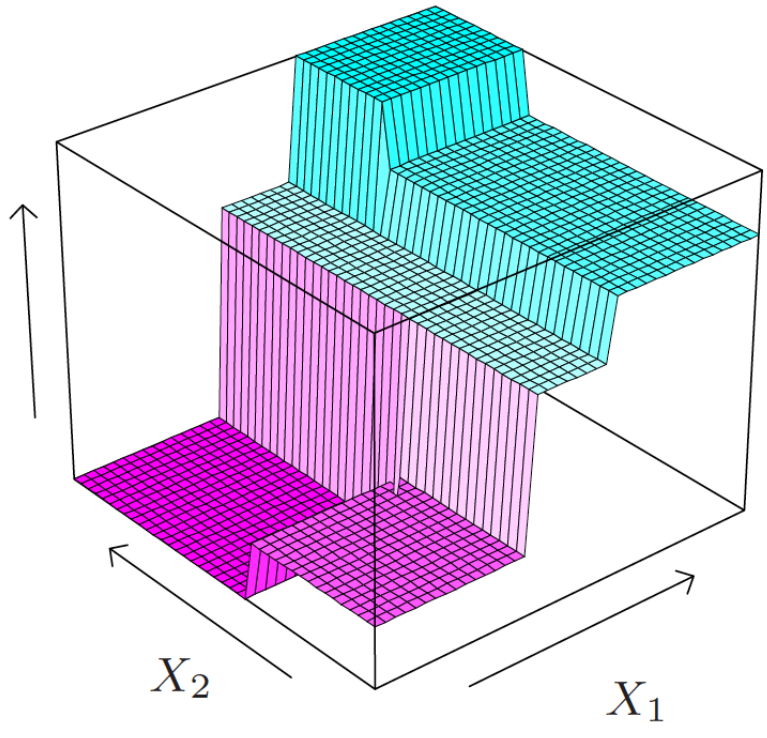
Other impurity measures



Regression trees

Suppose that now we want to predict a continuous outcome instead of a class label. Essentially, everything follows as before, but now we just fit a constant inside each rectangle





The estimated regression function has the form

$$\hat{f}^{\text{tree}}(x) \;=\; \sum_{j=1}^{m} c_j \cdot 1\{x \in R_j\} \;=\; c_j \text{ such that } x \in R_j$$

just as it did with classification. The quantities c_j are no longer predicted classes, but instead they are real numbers: the average response within each region:

$$c_j = \frac{1}{n_j} \sum_{x_i \in R_j} y_i$$

The main difference in building the tree is that we use squared error loss instead of misclassification error (or Gini index or deviance) to decide which region to split.

Categorical predictors

Awesome properties!

Trees have some amazing properties, especially when compared to other methods we've looked at.

- The ability to tune bias/variance!
- Interpretability!
- Automatic handling of variable transformations!
- Automatic detection of interactions!

How well do trees predict?

Trees have so many amazing properties! Unfortunately. . . they don't usually predict well. . .

Trees tend to suffer from high variance for a given amount of flexibility because they are quite unstable: a smaller change in the observed data can lead to a dramatically different sequence of splits, and hence a different prediction.

This instability comes from their hierarchical nature; once a split is made, it is permanent and can never be "unmade" further down in the tree

Salvaging Trees

Trees have many incredibly useful properties, but don't usually make great predictors.

Instead, we will use trees as building blocks in the construction of more complex, powerful predictive methods.

Our goal is to preserve as many of the useful tree properties as possible while dramatically improving our predictive ability.

Reducing the variance

We've seen that decision trees are very flexible, but have high variance. This leads to poor classification error.

How can we reduce variance?

We know that averaging independent things tends to reduce variance. But independent trees would require more data!

How could we achieve something similar?

The bootstrap

The bootstrap[2] is a fundamental resampling tool in statistics.

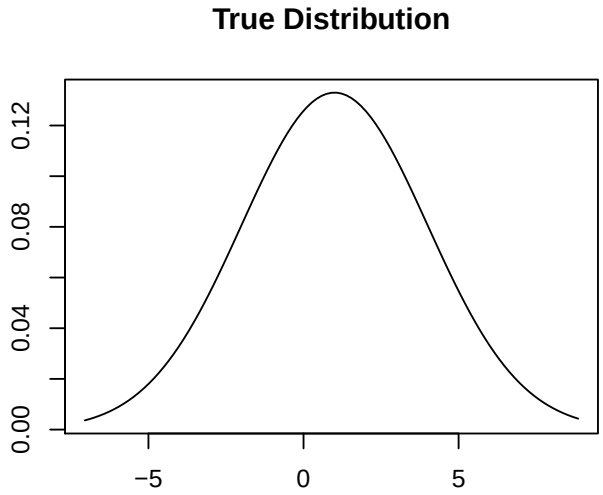
The basic idea underlying the boostrap is that we can estimate the true distribution $\mathcal{F}$ by the so-called empirical distribution $\hat{\mathcal{F}}$

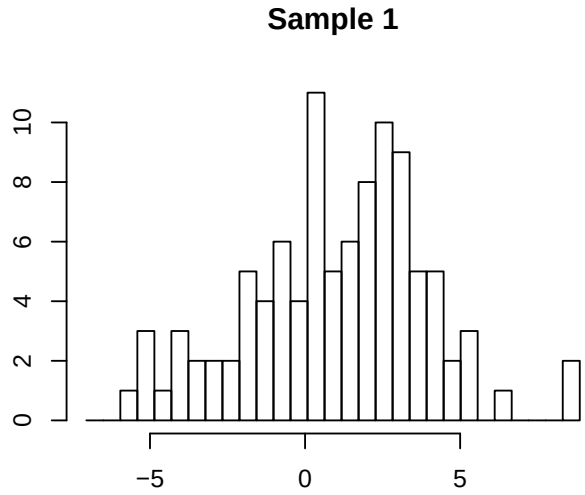
Given the training data (x_i, y_i), $i = 1, \ldots n$, the empirical distribution function $\hat{\mathcal{F}}$ is simply

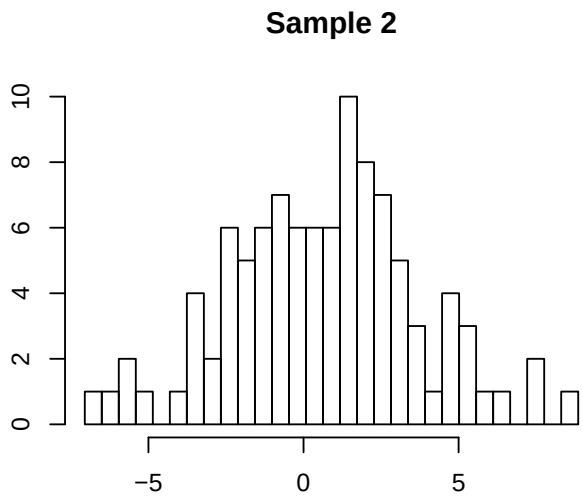
$$P_{\hat{\mathcal{F}}}\{(X, Y) = (x, y)\} = \begin{cases} \frac{1}{n} & \text{if } (x, y) = (x_i, y_i) \text{ for some } i \\ 0 & \text{otherwise} \end{cases}$$

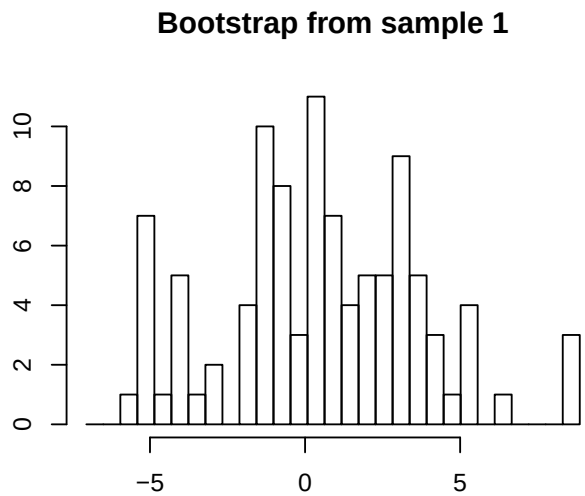
This is just a discrete probability distribution, putting equal weight $(1/n)$ on each of the observed training points

[2] Efron (1979), "Bootstrap Methods: Another Look at the Jacknife"









With the bootstrap, we are approximating the true distribution by a discrete distribution over the original sample data points.

A bootstrap sample of size m from the training data is

$$(x_i^*, y_i^*),\ i = 1, \ldots m$$

where each (x_i^*, y_i^*) are drawn from uniformly at random from $(x_1, y_1), \ldots (x_n, y_n)$, with replacement

This corresponds exactly to m independent draws from $\hat{\mathcal{F}}$. Hence it approximates what we would see if we could sample more data from the true $\mathcal{F}$. We often consider $m = n$, which is like sampling an entirely new training set

What shows up?

Note: not all of the training points are represented in a bootstrap sample, and some are represented more than once. When $m = n$, about 36.8% of points are left out, for large n (Try to prove this).

These left out points feel almost like a validation set. . . (Later)

Bagging

Bootstrapping gives us an approach to variance reduction!

We are worried that our tree could change dramatically if we drew a slightly different sample.

What if we draw many bootstrap data sets, fit a new tree on each one, and "average" their predictions (somehow)! Now we'll see all the different trees that might show up, and their combination will be stable!

You are likely accustomed to seeing the boostrap for estimating errors. Now we're using it to construct predictions!

Bagging (more carefully)

Given a training data (x_i, y_i), $i = 1, \ldots n$, bagging[3] averages the predictions from predictors (here trees) over a collection of boostrap samples.

For $b = 1, \ldots B$ (e.g., $B = 100$), we draw n boostrap samples (x_i^{*b}, y_i^{*b}), $i = 1, \ldots n$, and we fit a classification tree $\hat{f}^{\text{tree},b}$ on each sampled data set.

To classify an input $x \in \mathbb{R}^p$, we simply take the most commonly predicted class:

$$\hat{f}^{\text{bag}}(x) = \underset{k=1,\ldots K}{\text{argmax}} \sum_{b=1}^{B} 1\{\hat{f}^{\text{tree},b}(x) = k\}$$

This is just choosing the class with the most votes.

[3] Breiman (1996), "Bagging Predictors"

Tree depth

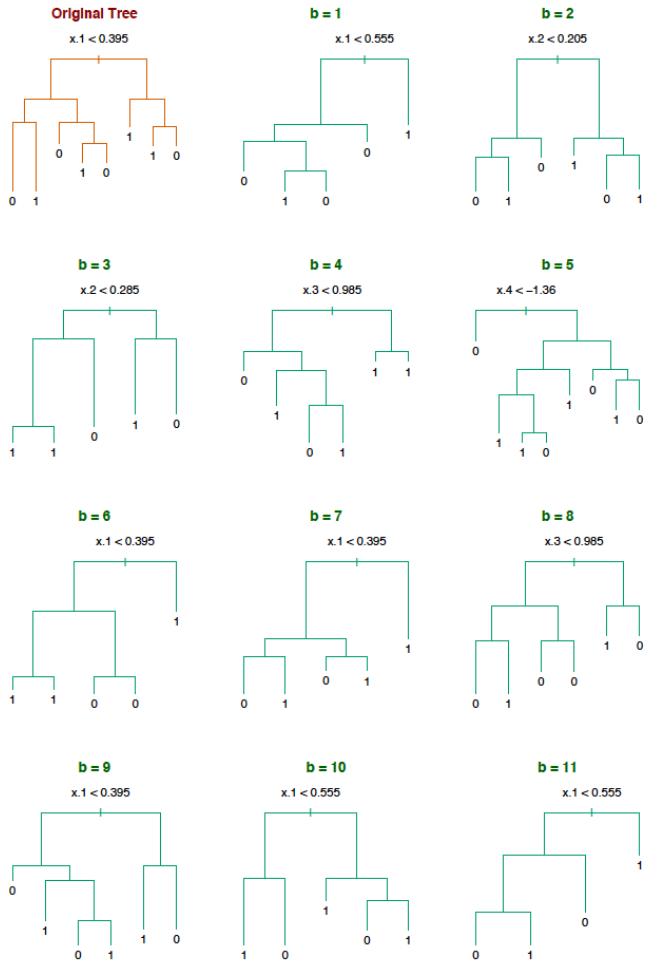
Most commonly, trees are grown fairly large on each sample, with no pruning. Why are we less worried about tuning each tree?

What happens to variance as we average many things together?

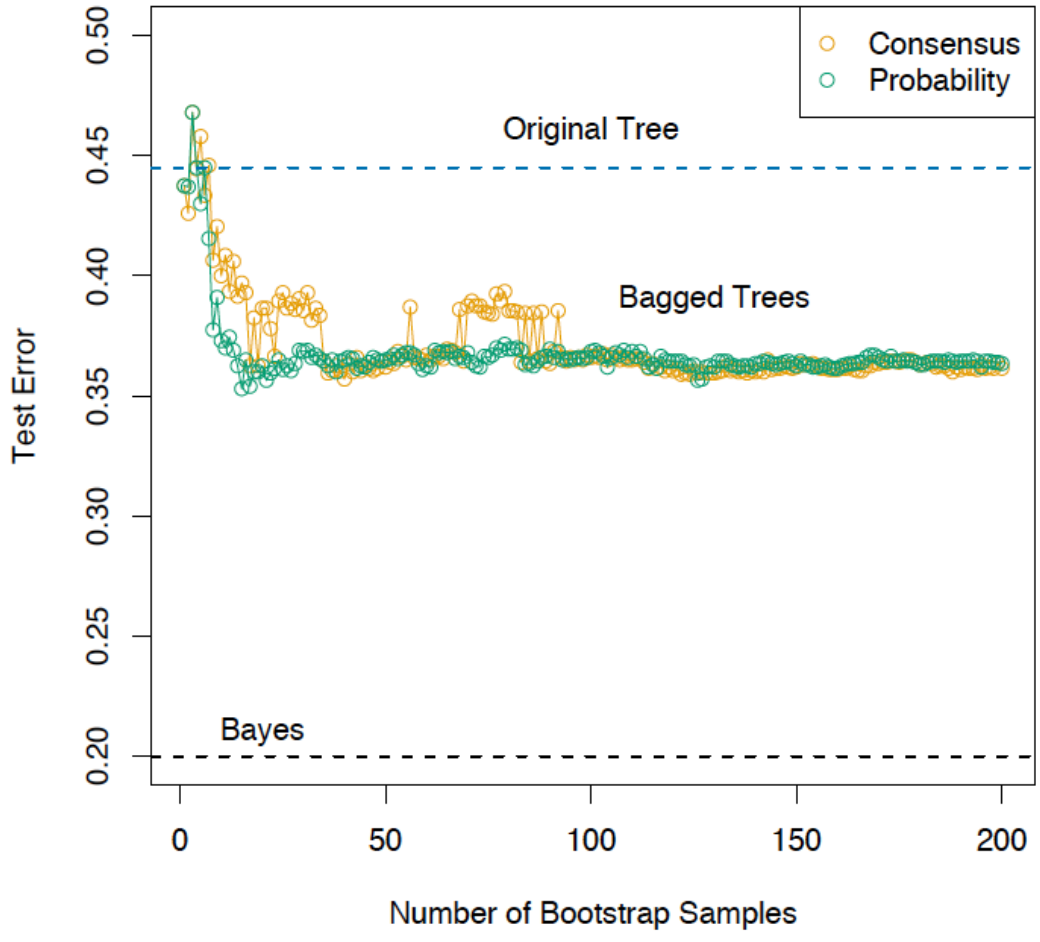
What happens to bias as we average many things together?

Example: bagging

Example (from ESL 8.7.1): $n = 30$ training data points, $p = 5$ features, and $K = 2$ classes. No pruning used in growing trees:



Bagging helps decrease the misclassification rate of the classifier (evaluated on a large independent test set). Look at the orange curve:



Voting probabilities are not estimated class probabilities

Suppose that we wanted estimated class probabilities out of our bagging procedure.

What about using the proportion of votes that were for class k?

$$\hat{p}_k^{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} 1\{\hat{f}^{\text{tree},b}(x) = k\}$$

This is generally not a good estimate.

Simple example: suppose that the true probability of class 1 given x is 0.75. Suppose also that each of the bagged classifiers $\hat{f}^{\text{tree},b}(x)$ correctly predicts the class to be 1. Then $\hat{p}_1^{\text{bag}}(x) = 1$, which is wrong

Estimated class probabilities

What's nice about trees is that each tree already gives us a set of predicted class probabilities at x:

$$\hat{p}_k^{\text{tree},b}(x),$$

These are simply the proportion of points in the appropriate region that are in each class

This suggests an alternative method for bagging. Now given an input $x \in \mathbb{R}^p$, instead of simply taking the prediction $\hat{f}^{\text{tree},b}(x)$ from each tree, we go further and look at its predicted class probabilities $\hat{p}_k^{\text{tree},b}(x)$, $k = 1, \ldots K$, and combine those!

Alternative form of bagging

We define the bagging estimates of class probabilities:

$$\hat{p}_k^{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{p}_k^{\text{tree},b}(x) \quad k = 1, \ldots K$$

The final bagged classifier just chooses the class with the highest probability:

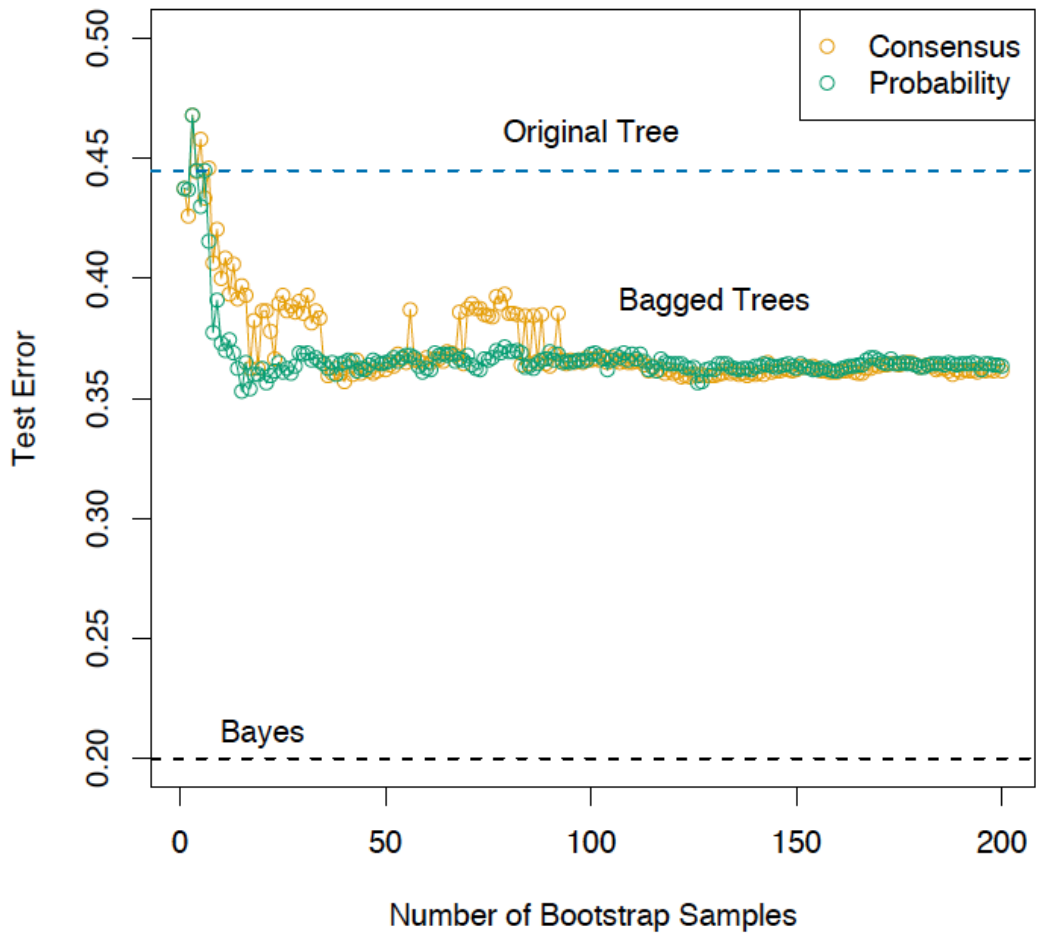
$$\hat{f}^{\text{bag}}(x) = \underset{k=1,\ldots K}{\text{argmax}}\ \hat{p}_k^{\text{bag}}(x)$$

This form of bagging is preferred if it is desired to get estimates of the class probabilities.

It also tends to give better predictions! Why?

Example: alternative form of bagging

Previous example revisited: the alternative form of bagging produces misclassification errors shown in green



Why is bagging working?

Why is bagging working? Let's consider a simplified setup. Suppose that we have $K = 2$ classes and B *independent* classifiers $\hat{f}^b(x)$, and that each classifier has a misclassification rate of 0.4.

The bagged classifier estimates

$$\hat{f}^{\text{bag}}(x) = \underset{k=0,1}{\text{argmax}} \sum_{b=1}^{B} 1\{\hat{f}^b(x) = k\}$$

Suppose that the correct class for x is 1. Let $V = \sum_{b=1}^{B} 1\{\hat{f}^b(x) = 1\}$ be the number of votes for class 1.

In this idealized story, the distribution of V is

V is $\mathrm{Binomial}(\mathrm{B}, 0.6)$, and we can write the misclassification rate for the bagged classifier as

$$\mathrm{P}(\hat{f}^{\mathrm{bag}}(x) = 0) = \mathrm{P}(V \leq B/2).$$

As $B \to \infty$, $P(V \leq B/2)$.

So why did the prediction error seem to level off in our examples?

Random Forests: Improved Bagging

Random forests[4], an incredibly popular and successful prediction approach, are a simple modification of bagged trees.

We've seen that bagging should improve with independence of the trees.

To improve bagging, we want to make our trees more independent while using the same data.

Imagine bagging when you have a few strong variables. The same variables can dominate the splits across your trees, highly correlating their predictions.

[4] Breiman (2001), "Random Forests"

Random Forests: Improved Bagging

To make the trees more independent, we only allow a small subset of variables to be considered at each split!

At each split, $m < p$ variables are selected, and a split is chosen from among those variables

This makes the trees more varied and less independent, at the cost of making our individual greedy optimizations *worse!*

However, the result is a model with much better predictions, and a nice balance among correlated variables.

How do we estimate error?

How can we estimate how well we will predict/classify on new data?

Cross-validation is tempting, because we have no creativity.

However, cross-validation could be quite expensive, since we're boostrapping and developing hundreds of trees each time.

What about the samples each bootstrap did not select?

Out-of-Bag error estimation

Each training point should be missing from about $1/3$ of the bootstrap samples. That subset of the trees have not been overfit to the training point!

For each sample point, we get an average prediction from the $1/3$ of the trees that didn't include it. We combine these predictions over all the points to get an out-of-bag MSE estimate.

This acts like a cross-validated estimate, but it's free!

Random forest properties

Trees have some amazing properties, especially when compared to other methods we've looked at.

- Surprisingly impressive predictive performance!
- Automatic handling of variable transformations!
- Automatic detection of interactions!
- What about Interpretability?

Digging into Random Forests

The bagged classifier is now an average of many trees. This is much harder to interpret! That was half the reason we loved trees!

Similarly, one of the big selling points of the Lasso is that it produces *interpretable* models. By seeing the sparsity pattern, we know which variables are being used for predicting.

We see that random forests predict incredibly well. However, an average of hundreds of random trees is harder to understand.

We need tools to peek into these "black box" methods to understand how they're using the data.

Variable Importance

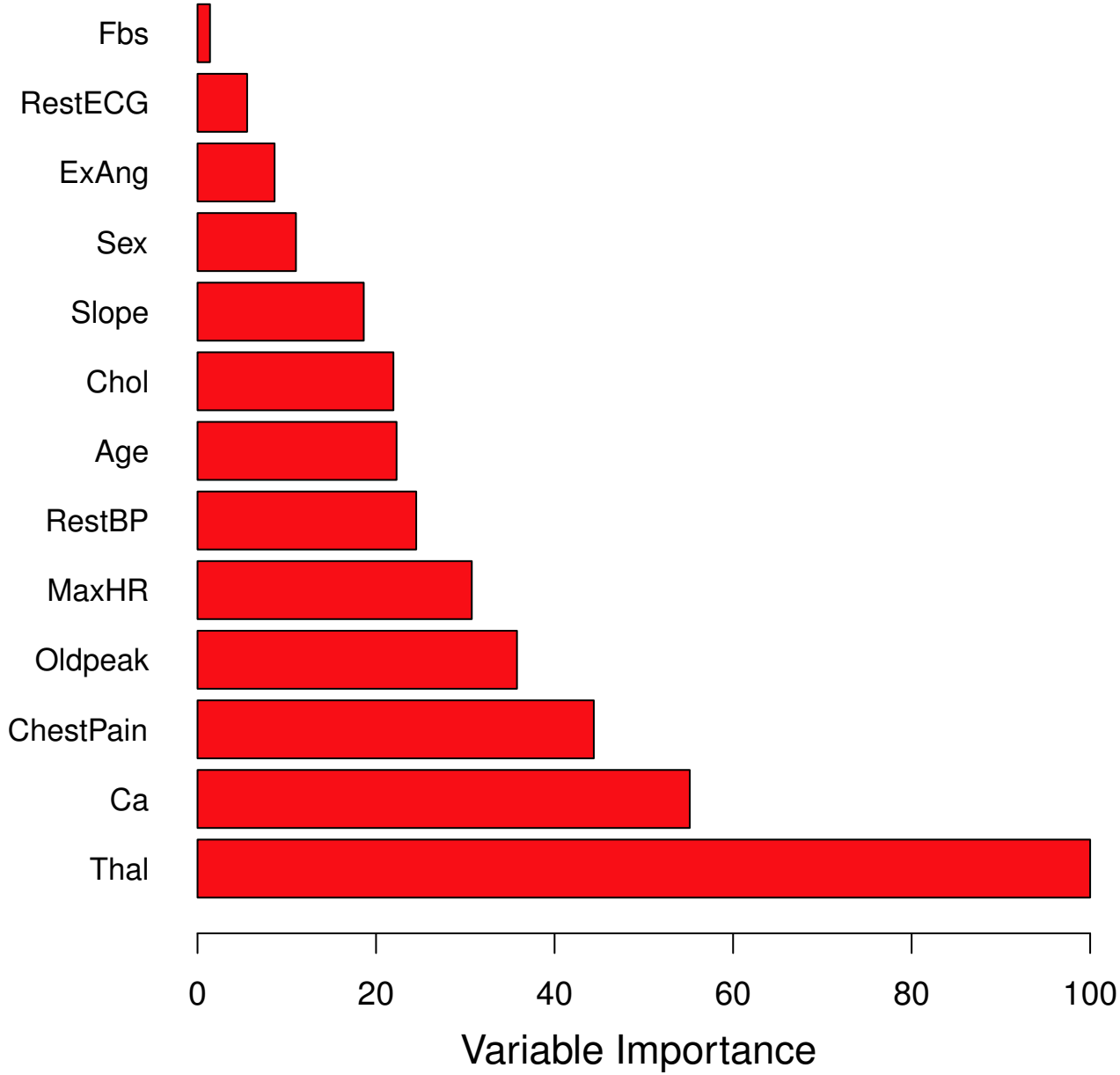
The most basic task is to understand which predictors are important for making the random forest prediction.

Random forests don't explicitly do variable selection like the lasso, but their individual trees tend to rely more on informative variables when they search for the next split.

There are two popular ways to measure variable importance:

1. For each variable, measure the amount that RSS (or Gini index) decreases due to splits in that variable. Average this over all trees in the forest
2. Randomly permute each variable (one at a time) and see how much the model performance decreases.

The `varImpPlot` method in `R` computes the first of these.


Fbs
RestECG
ExAng
Sex
Slope
Chol
Age
RestBP
MaxHR
Oldpeak
ChestPain
Ca
Thal
0
20
40
60
80
100
Variable Importance

Partial dependence plots

Variable importance tells you which variables are useful, but not how they are used.

Partial dependence plots can give some insight into the way that estimates change with each variable.

Suppose we divide our variables into sets $\mathcal{S}$ and $\mathcal{C}$, and we want to assess the effect of the variables in $\mathcal{S}$ on our predictions.

The partial dependence of $f(X)$ on $X_{\mathcal{S}}$ is defined as

$$f_{\mathcal{S}}(X_{\mathcal{S}}) = \mathbb{E}_{X_{\mathcal{C}}} f(X_{\mathcal{S}}, X_{\mathcal{C}})$$

This can be estimated by

$$\bar{f}(X_{\mathcal{S}}) =$$

where $x_{i\mathcal{C}}$ are just the values that appear in our data.

Partial dependence

Partial dependence works out to be a nice definition for simple models.

Suppose the truth is that $f(X) = f_1(X_1) + f_2(X_2)$. Then

$$\bar{f}(X_1) = \mathbb{E}_{X_2} f(X_1, X_2) = \mathbb{E}_{X_2} \left(f_1(X_1) + f_2(X_2) \right)$$
$$= $$

Suppose the truth is that $f(X) = f_1(X_1) \cdot f_2(X_2)$. Then

$$\bar{f}(X_1) = \mathbb{E}_{X_2} f(X_1, X_2) = \mathbb{E}_{X_2} \left(f_1(X_1) \cdot f_2(X_2) \right)$$
$$= $$

Disadvantages

It is important to discuss some disadvantages of bagging:

- *Loss of interpretability:* the final bagged classifier is not a tree, and so we forfeit the clear interpretative ability of a classification tree

- *Computational complexity:* we are essentially multiplying the work of growing a single tree by B (especially if we are using the more involved implementation that prunes and validates on the original training data)